Scaling E-commerce Analytics

Distributed Sessionization & Attribution using Spark

Dax Rajani

40294118

Harsh Ahuja

40301748

Submitted to: Grengrui Zhang

Problem Statement

Challenge

Processing 42M+ behavioral events

Bottleneck

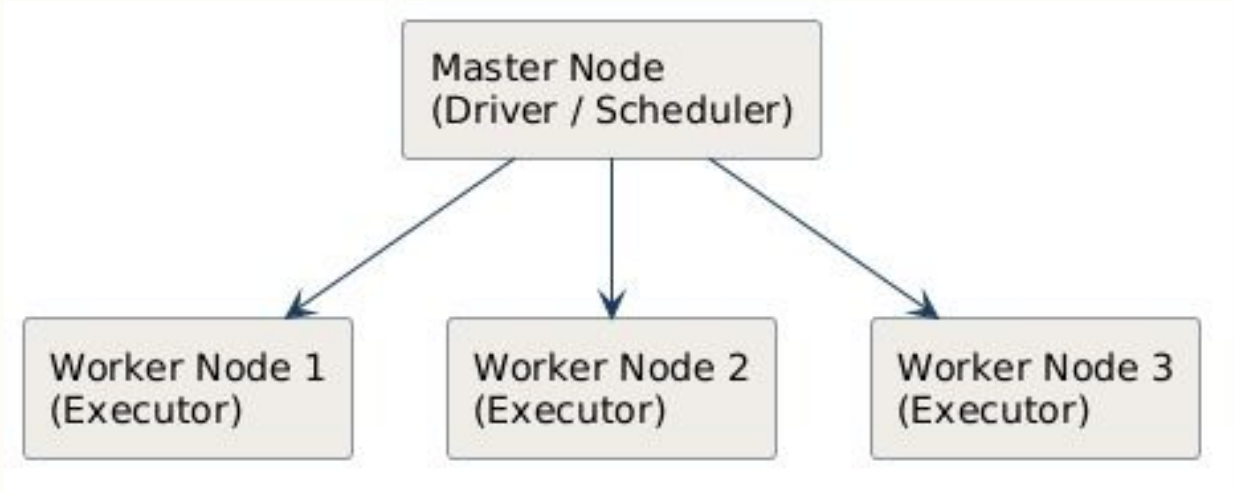
High RAM requirement & operating hours by traditional tools

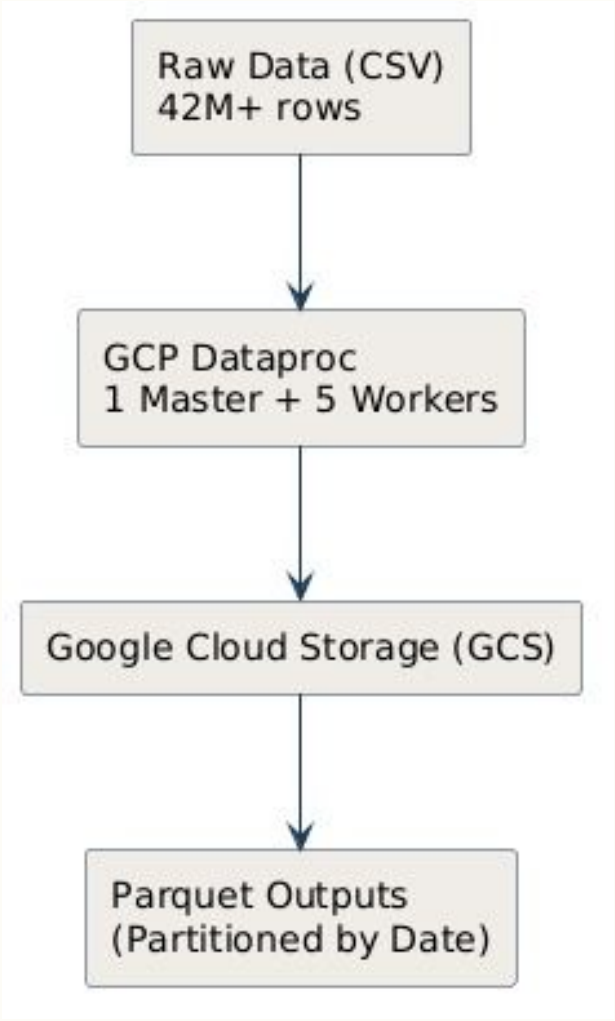
Objective

Build a distributed & Scalable pipeline for complete e-commerce analytics

System Architecture & Data Flow

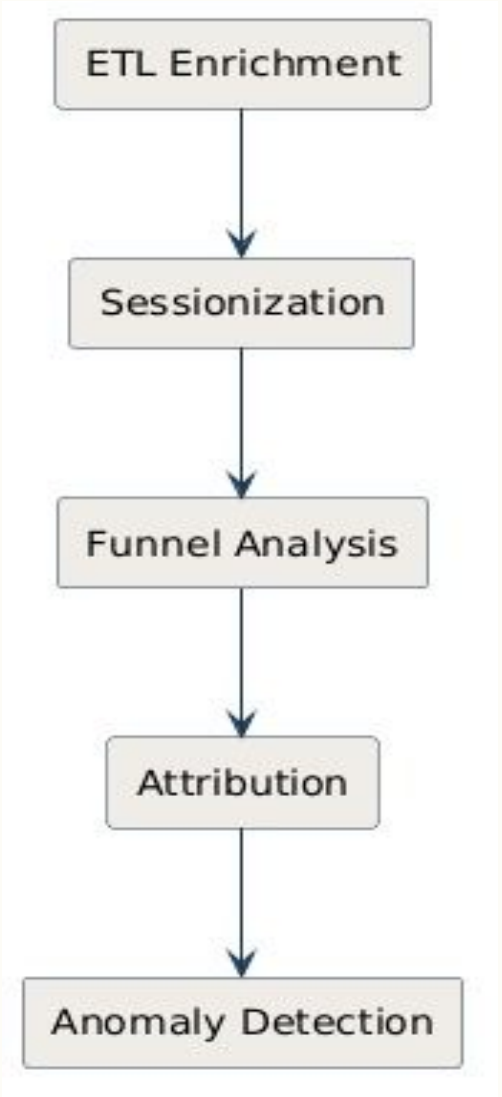
- **Engine:** Apache Spark (PySpark)
- **Cloud Infrastructure:** GCP Dataproc (1 Master, up to 5 Workers)
- **Storage**: Google Cloud Storage (GCS)
- **Data Format:** Apache Parquet





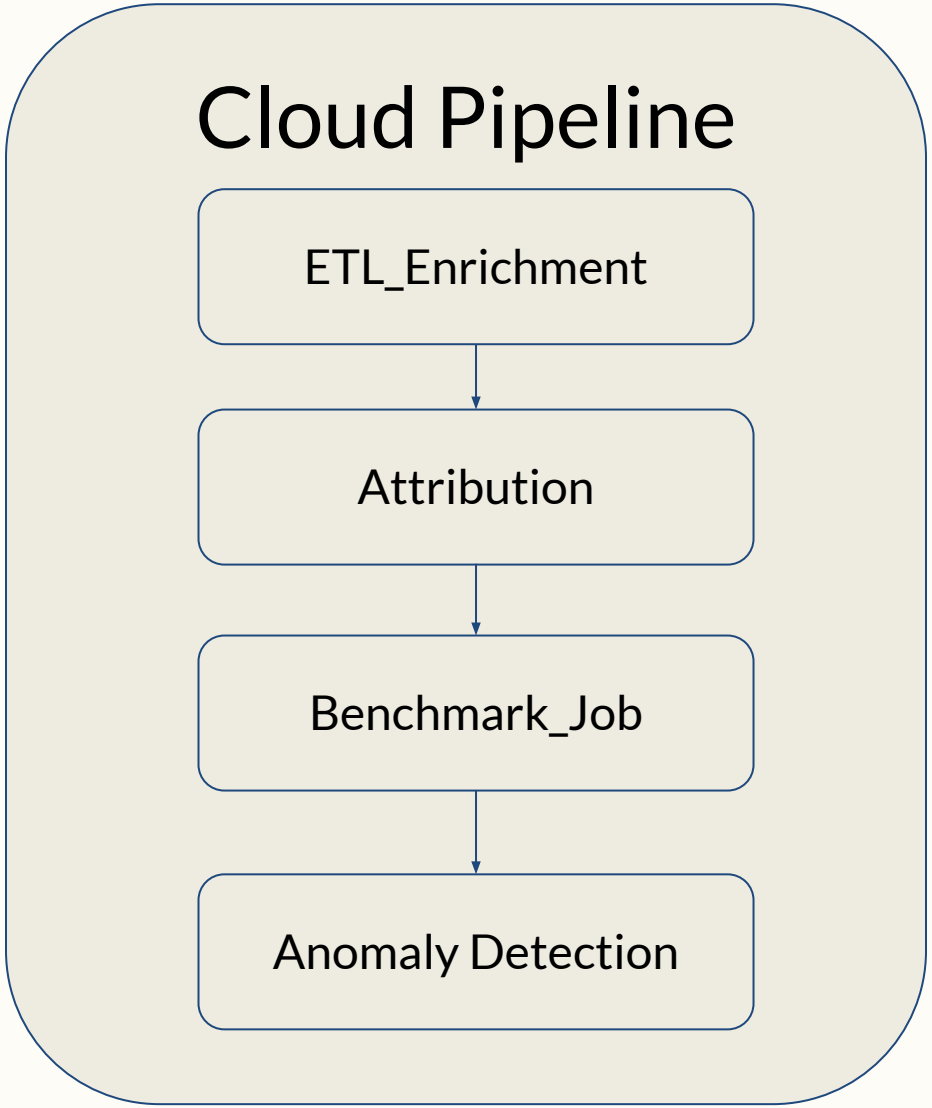
Implementation & Modules

- **Stage 1(ETL) :** Synthesizes missing fields and writes Parquet outputs
- **Stage 2 (Sessions):** Groups 30-min user sessions
- **Stage 3 (Funnels):** Calculates view→cart→purchase conversion rates
- **Stage 4 (Attribution):** Credits the last non-direct referrer.
- **Stage 5 (Anomalies):** Flags spikes/drops using a 7-day trailing rolling z-score.



Implementation Summary & Codebase Modules

- **Development:** Leverages PySpark DataFrames and the Catalyst optimizer for automated query tuning.
- **Execution Model:** Builds a 5-stage DAG via lazy evaluation, triggering only on action.
- **Deployment:** Deployed as a unified script to GCP Dataproc to bypass complex dependency management.



Strengths & Weaknesses

Strengths

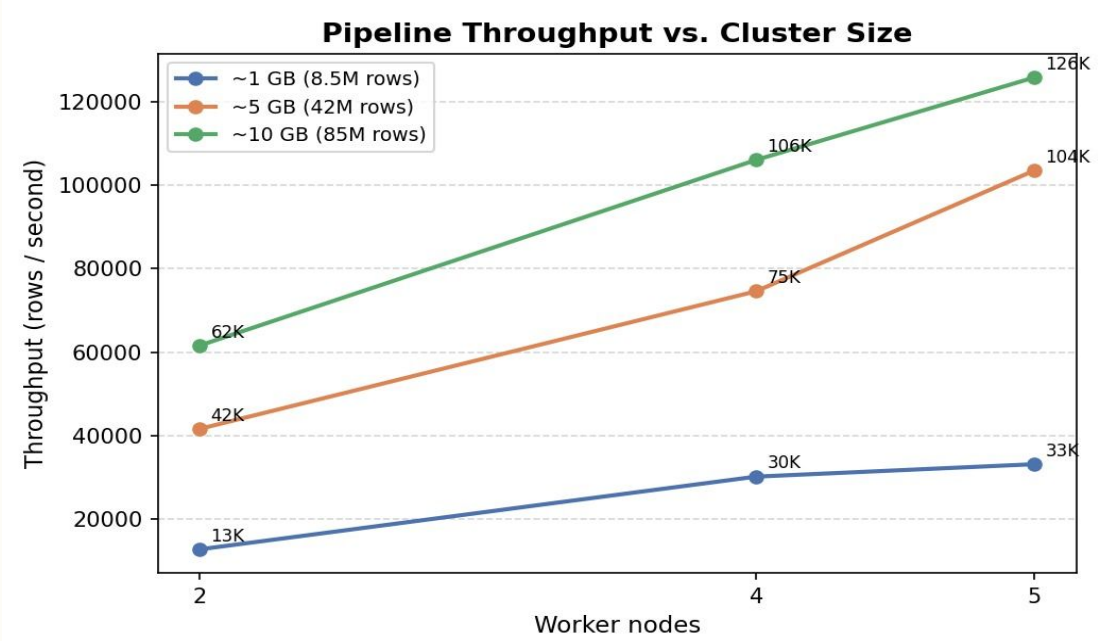
- **High Data Compression:** Reduced the dataset from 5.3 GB to ~1.2 GB, achieving 4.4× compression.
- **Reproducibility:** Running the pipeline twice produces the exact same device and referrer assignments for accurate benchmarking.

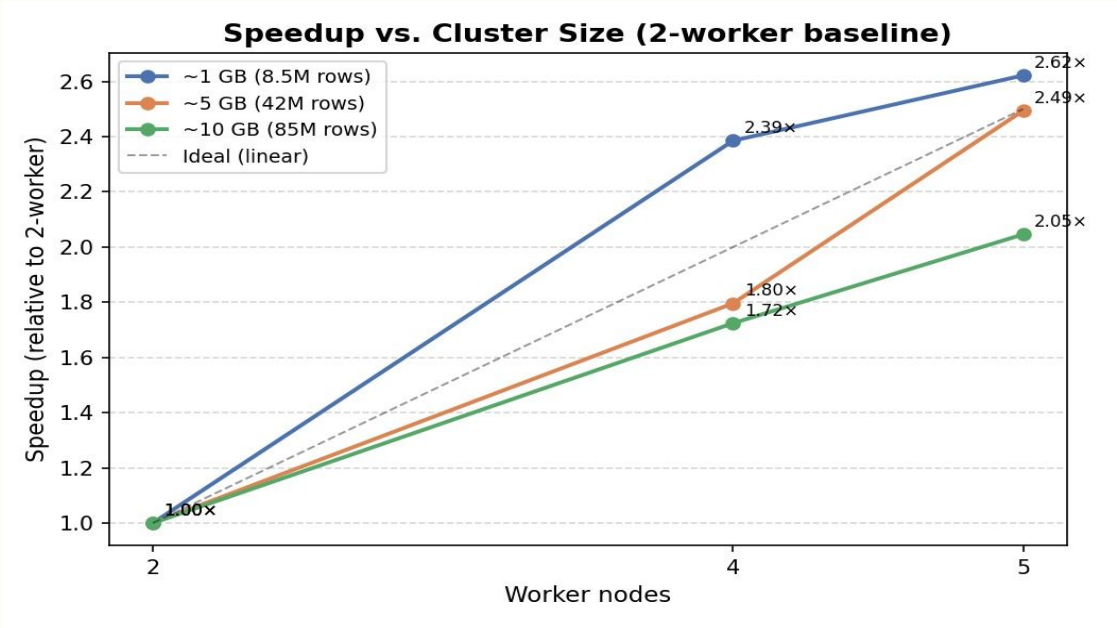
Limitations

- **I/O Bottlenecks:** Writing the Parquet tables to GCS took 477 seconds, consuming 35% of the total pipeline runtime on a 10 GB run.
- **Sub-linear Scaling:** observed 2.05× vs. ideal 2.5× on large data

Workload Scaling

- **Test Environment:** Benchmarked across 2, 4, and 5 worker nodes using 1 GB, 5 GB, and 10 GB data samples.
- **Overall Runtime:** Processing the full 10 GB dataset dropped from 1,378.9 seconds (2 workers) to 674.0 seconds (5 workers).
- **Throughput Gains**: System scaled from 12,607 rows/sec (2 workers, 1 GB) to a peak of 125,961 rows/sec (5 workers, 10 GB).
- **Task Efficiency:** The 1 GB dataset achieved super-linear speedup (2.62×) due to optimal memory fit and low task scheduling overhead.





Performance Optimization

- **Broadcast Joins:** Bypassed expensive network shuffles via Broadcast joins, reducing join time by 1.58× (11.33s to 7.15s).

- **Partition Pruning:** Achieved a 3.11× speedup (4.25s to 1.37s) on single-day queries by organizing Parquet outputs into date partitions.